Introducción a Git 🕰️

`Git` es el **sistema de control de versiones distribuido** más utilizado en la industria del software. Nos permite registrar, rastrear y gestionar todos los cambios realizados en el código fuente de un proyecto.

Fue creado por **Linus Torvalds** (*el mismo que desarrolló el núcleo de Linux*) con el propósito de ser rápido, seguro y eficiente, especialmente para proyectos con muchos integrantes.

Analogía 🔍

Imagina que estás escribiendo un libro en equipo con otras personas. Cada quien trabaja en su propio capítulo. `Git` actúa como la persona a cargo de:

- Guardar cada versión del libro.
- Indicar quién escribió qué y cuándo.
- Volver a versiones anteriores si algo salió mal.
- Combinar cambios de distintas personas sin borrar el trabajo de nadie.

Es, literalmente, una **máquina del tiempo para tu código** ⏳



¿Qué problema resuelve Git? 🛠️

Antes de `Git`, era común usar copias de carpetas con nombres como:

`proyecto-final-v1`

`proyecto-final-v1-REAL`

`proyecto-final-v1-DE-VERDAD-AHORA-SÍ`

Este tipo de manejo de versiones es propenso a errores, difícil de mantener y completamente manual. Seguro que te has visto en una situación similar con los ejercicios que has desarrollado 👀.

Acá es donde entre en juego `Git`, que nos permite mantener diferentes versiones, regresar a versiones anteriores y colaborar con otras personas de forma sencilla.

Sistemas de control de versiones 🗓️

Un `VCS` (*Version Control System*), es una herramienta que:

- Guarda un historial de todos los cambios realizados en el código.
- Permite volver a versiones anteriores sin perder avances.
- Evita métodos rudimentarios como duplicar carpetas con nombres como `Proyecto-FINAL-FINAL`.

Aunque existen muchos `VCS`, `Git` se ha vuelto el estándar de facto en la industria del software. Casi todos los proyectos en plataformas como `GitHub`, `GitLab` o `Bitbucket` están gestionados con `Git`.

Repositorios de Git 🏠

Un `repositorio` es el espacio donde vive tu proyecto de código, junto con su historial de cambios.

Es el contenedor de todo lo que haces con `Git`.

Existen dos tipos de repositorio:

- 🚪 **Repositorio local**: está en tu computadora.
- 🌐 **Repositorio remoto**: está en la nube, por ejemplo, en `GitHub`.

Para trabajar con `Git`, primero creamos el repositorio local y luego lo conectamos con uno remoto.

Git vs GitHub vs GitLab 🧩

A lo largo de tu camino como desarrollador, vas a escuchar una y otra vez los términos `Git`, `GitHub` y `GitLab`. Al principio puede parecer que todos son lo mismo, pero en realidad cada uno cumple un rol diferente.

`Git` es la herramienta de control de versiones. Te permite registrar y gestionar los cambios en tu código de manera local. Es como tener una libreta donde anotas cada cambio que haces.

`GitHub` es una plataforma en línea que te permite subir tus repositorios `Git` para compartirlos, colaborar con otros y gestionar proyectos.

`GitLab` es otra plataforma para trabajar con `Git`, pero a diferencia de `GitHub`, es de código abierto y muchas empresas la instalan en sus propios servidores. Ideal para quienes quieren mantener el control total sobre su infraestructura.

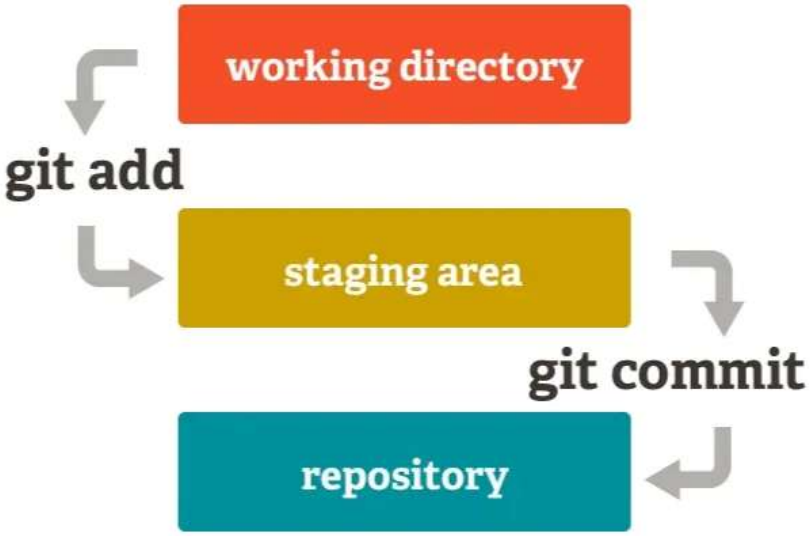
> Ofrece funcionalidades similares a `GitHub`: control de versiones, gestión de incidencias, integración y entrega continua (CI/CD), entre otras.

🧠 `Git` es el libro, `GitHub` y `GitLab` son las bibliotecas donde lo almacenas y prestas a otros.

Flujo Básico de Git

Cuando trabajamos con `Git`, cada archivo que modificamos pasa por distintas **etapas** antes de quedar registrado en el historial del proyecto.

Conocer este flujo es fundamental para entender a profundidad cada uno de los comandos que vamos a utilizar al trabajar con nuestro repositorio.



Paso 1: Working Directory (*Directorio de trabajo*) 🛠️

Antes de empezar a registrar cambios, necesitamos inicializar un proyecto para que `Git` comience a rastrear nuestros archivos.

📌 **Comando para crear el repositorio:**

```
1  git init
```
Copiar

Esto crea una carpeta oculta llamada `.git`, que es donde se guardará toda la información sobre el historial de los proyectos. Este paso **convierte cualquier carpeta** en un repositorio de `Git`. A partir de ahí, puedes comenzar a trabajar con control de versiones localmente.

Una vez tengamos nuestro Working Directory creado, podemos empezar a trabajar en los cambios al código con tu editor de código (*como Visual Studio Code*).

Todos los archivos nuevos, modificados o eliminados comienzan su recorrido aquí. `Git` detecta estos cambios, pero **todavía no los guarda** en su historial.

Comando `git status` 👨🏻‍💻

Este comando te muestra en qué estado se encuentran tus archivos dentro del proyecto.

Es nuestra herramienta principal para **verificar qué está pasando** en el repositorio en cualquier momento. Con este comando puedes saber:

Qué archivos cambiaron.

Cuáles están listos para guardar.

Si hay archivos nuevos sin seguimiento (`untracked`).

Entonces, una vez hayamos realizado cambios en el repositorio, vamos a ejecutar:

```
1  git status
```
Copiar

Paso 2: Staging Area (*Área de preparación*) 🧺

Acá vamos a decidir los archivos que vamos a incluir en el próximo `commit`. Es una bandeja donde vamos a colocar solo lo que vamos a guardar. Entonces, una vez queramos agregar archivos al staging ejecutamos uno de los siguientes comandos:

📌 **Mover un único archivo al `staging`:**

```
git add práctica_semana_1.txt
```

Copiar

📌 **Mover todos los archivos modificados al `staging`:**

```
git add .
```

Copiar

Paso 3: Local Repository (*Repositorio local*) 📚

En este paso, vamos a registrar en el historial del repositorio los cambios de los archivos que preparamos en el paso anterior.

Para ello vamos a realizar un `commit`, lo que captura el estado actual del repositorio y crea un identificador único junto con un mensaje descriptivo que agreguemos.

Esto te permite:

Recuperar versiones anteriores del código.

Saber quién cambió qué.

Dejar mensajes que expliquen por qué se hicieron esos cambios.

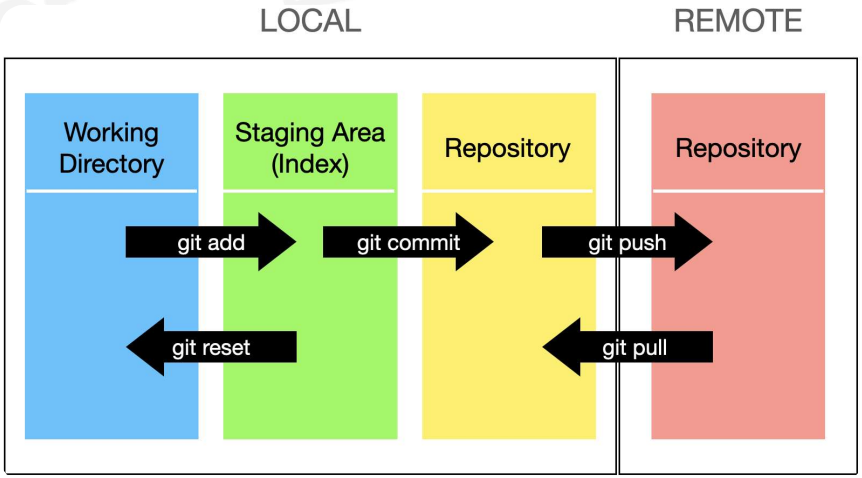
📌 **Crear un commit con mensaje:**

```
git commit -m "Agrega ejercicios de práctica de semana 1 a 3"
```
Copiar

Un `commit` es **una foto del estado del proyecto**, acompañada de una nota que dice *"esto fue lo que cambió y por qué"*.

Publicar repositorio para compartir el código 🌐

Para que tu trabajo en `Git` esté respaldado y actualizado tanto en tu máquina como en la nube, necesitas mantener sincronizados el repositorio **local** y el **remoto**.



Sincronizar desde local hacia remoto 🔄

Para lograrlo, cuando quieras publicar una nueva versión de tu código en `GitHub` o `GitLab` y ya hiciste los `commits` puedes ejecutar el siguiente comando:

```
git push origin nombre-de-la-rama # Cualquier nombre que quieras poner a la rama
```

Copiar

Esto sube los cambios de tu rama local al repositorio **remoto**.

Sincronizar desde remoto hacia local 🏠

Antes de empezar a trabajar, es buena práctica traer los cambios más recientes que hayan hecho otros. Para ello vamos a sincronizar los cambios que estén en el repositorio remoto con el siguiente comando:

```
git pull origin nombre-de-la-rama
```

Copiar

Esto evita conflictos y asegura que trabajas con la versión más actual del proyecto.

Trabajando con ramas

Una de las funcionalidades más poderosas de `Git` es la posibilidad de trabajar múltiples versiones de código nuestro de forma aislada, ordenada y segura.

Esto permite que distintos desarrolladores (*o nosotros mismos*) trabajen en funcionalidades nuevas, correcciones o pruebas **sin afectar directamente** el proyecto principal.

¿Qué es un branch?

Un `branch` (*rama*) es una **línea paralela de trabajo** que parte de una versión del código y permite hacer cambios sin alterar el original. Cuando los cambios están listos, podemos unir esa rama con otra mediante un proceso llamado `merge`.

📘 Piensa en las `ramas` como caminos alternativos donde puedes experimentar, construir nuevas ideas o corregir errores. Luego puedes decidir si esos cambios se integran al camino principal (`main`) o no.

Comandos para trabajar con branches

Crear una nueva rama y cambiara ella

```
1  git checkout -b nueva-rama
```

Copiar

Cambiar de rama:

```
1  git checkout nombre-de-la-rama
```

Copiar

Listar ramas disponibles en el repositorio

```
1  git branch
```

Copiar

Combinar ramas con merge

Una vez que terminas de trabajar en una `branch`, puedes **unir sus cambios** con otra rama. Este proceso se llama `merge`.

¿Qué es un `merge`?

Hacer un `merge` significa combinar los cambios de una `branch` secundaria dentro de otra rama principal, generando una versión unificada del proyecto.

📘 Piensa en un `merge` como **juntar dos caminos en uno solo**, manteniendo todos los avances.

¿Cómo hacer un merge? 🛠️

Opción 1: Manual desde consola 🔶

Para hacer un `merge` desde la terminal, primero debes estar **ubicado en la rama que va a recibir los cambios** (por ejemplo, `main`), y luego ejecutar el comando `git merge`.

```
1  git checkout main     # Nos ubicamos en la rama principal
2  git merge nueva-rama  # Traemos los cambios desde 'nueva-rama'
```

Copiar

Este proceso **no borra** la `branch` original (`nueva-rama`). Solo une sus cambios a la rama actual.

Opción 2: Pull Request desde GitHub 🔷

Cuando trabajamos en equipo, lo más común es hacer el `merge` a través de una `Pull Request`. Esto permite revisar, discutir y aprobar los cambios antes de integrarlos. Para lograrlo puedes seguir estos pasos:

11. Ve a la pestaña `Pull requests` en tu repositorio.
12. Haz clic en **New pull request**.
13. Selecciona la `branch` de origen (la que tiene los cambios) y la de destino.
14. Escribe un título y descripción clara.
15. Solicita revisión de otros colaboradores.
16. Haz clic en **Merge pull request** cuando todo esté listo.

Flujo común de trabajo (*Git Flow*) 🔀

`Git Flow` es un flujo de trabajo estandarizado que define **cómo organizar las ramas** en un proyecto. Es utilizado para mantener procesos de desarrollo ordenados, seguros y colaborativos.

En `Lyfter`, utilizamos una versión **simplificada** este modelo. Aprender este flujo te prepara para entornos profesionales reales. 💼

Estructura principal de ramas 🌳

🌿 `main` (o `master`)

La rama `main` representa el **código en producción**. Solo contiene versiones **estables y verificadas** del proyecto. Es como el historial oficial de entregas.

- Cada commit en `main` es una versión que ya fue probada y lanzada.
- No se desarrolla directamente en esta rama.
- Cada ● es un commit (*una versión estable*).

🔧 `develop`

La rama `develop` se crea a partir de `main` y sirve como el **espacio de trabajo principal** para desarrolladores. Aquí se integran todas las nuevas funcionalidades antes de que estén listas para producción.

- Aquí se prueba todo antes de liberar una nueva versión.
- Es como un "*taller*" donde se ensamblan los cambios.
- La rama `develop` nace desde `main`.

✨ `feature`

Para trabajar en una nueva funcionalidad, como **crear un botón**, se crea una rama `feature/nombre` desde `develop`. Esto permite trabajar de forma aislada.

- Cuando el botón está listo y probado, se fusiona de vuelta a `develop`.
- Así no se interrumpe el trabajo de otros ni se rompe el proyecto.

Se crea la rama `feature/boton` desde `develop`.

Se hacen varios commits (`diseño`, `lógica`, `prueba`).

Al final, se hace un merge a `develop`.

🔁 ¿Cómo fluye el trabajo?

26. Se crea una rama `feature` desde `develop` para trabajar en algo nuevo.
27. Al finalizar la tarea, esa rama se **fusiona** (`merge`) de nuevo con `develop`.
28. Cuando `develop` ha acumulado suficientes funcionalidades, se hace `merge` a `main` para una nueva versión en producción.

💡 Ventajas de este flujo

Mantiene separadas las funcionalidades en desarrollo del código que está en producción.

Permite trabajo colaborativo sin riesgo de romper el proyecto principal.

Facilita el control de calidad y las pruebas antes de liberar cambios.

Comandos esenciales de Git 📝

Aquí tienes una lista de los comandos que más vas a usar en tu día a día como desarrollador.

¡Tómale captura a esta tabla! Ya que vas a necesitar tener a mano estos comandos. 📸

Comando	Descripción
`git init`	Inicia un repositorio local.
`git status`	Muestra archivos modificados.
`git add nombre-archivo`	Agrega archivos al stage.
`git commit -m "mensaje"`	Crea un commit con los archivos en el stage.
`git checkout -b nombre-rama`	Crea una nueva rama.
`git checkout nombre-rama`	Cambia de rama.
`git push`	Envía cambios al repositorio remoto.
`git pull`	Descarga cambios del repositorio remoto.

Conclusión 🎉

Dominar `Git` no es solo una habilidad técnica más. Es una herramienta **fundamental** que nos acompaña en toda nuestra carrera como desarrolladores.

No importa si estás construyendo un proyecto personal, colaborando en un equipo remoto o lanzando un producto a producción:

`Git` va a estar ahí, gestionando cada cambio, cada decisión, cada versión 👀

Repite. Practica. Equivócate. Aprende. 🔁

Recuerda que aprender `Git` lleva tiempo. Al inicio puede parecer complejo, pero cada vez que lo uses, incluso al cometer errores, estarás entrenando una habilidad que marcará una gran diferencia en tu carrera.

Lyfter

💡 **Todo gran desarrollador tiene una cosa en común: sabe usar `Git`.**

Y no "*más o menos*", sino que lo domina, con confianza, seguridad y claridad.